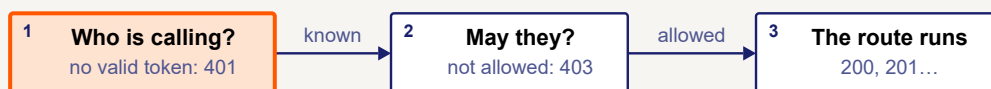


1. **Who are you, what may you do:** Authentication vs authorisation · Remembering who you are: sessions vs tokens
2. **Passwords:** Store a hash, never the password · Salt and cost: why bcrypt is slow on purpose
3. **Tokens and cookies:** Cookies and their security flags · Anatomy of a JWT · Signing and verifying: what the secret does · Register and log in, step by step · Sending and expiring tokens
4. **Protecting routes:** The auth middleware: one gate in front of many routes · Ownership and roles: a valid token is not a free pass
5. **CORS:** Origins and the same-origin policy · CORS: the server says who may read · Preflight requests · Cookies across origins: credentials · What CORS is not
6. **Hardening the API:** Validation with a schema: Zod and a validate middleware · Security middleware: helmet, rate limits, body limits, HTTPS · The signing secret: leaks and fallbacks · Pagination limits as protection · The security mistakes small APIs actually make

Who are you, what may you do

Authentication vs authorisation

Authentication answers "who is calling?"; **authorisation** answers "may this caller do this, to this resource?". Failing the first gives 401; failing the second gives 403.



Who is calling comes first; what they may do comes second.

The two questions

	Authentication	Authorisation
Question	Who is calling?	May this caller do this to this resource?
Evidence	A password at log-in, then a token or session cookie on every request	The user id and role from the verified token, plus the resource's owner
Where in an Express API	An auth middleware that sets req.user (see The auth middleware)	Each controller action, or a role middleware (see Ownership and roles)
On failure	401: missing, invalid or expired token	403: valid token, not yours or not allowed

Example. A Tasks API's GET /tasks/7: with no Authorization header → **401** Missing token; with an expired token → **401**; with Ana's valid token when task 7 belongs to Ben → **403**; with Ana's token for her own task 7 → **200**; with any valid token for task 999, which does not exist → **404**.

Common mistake. Answering 403 for a missing token, or 401 for "not your task". The client reacts to each code differently: on 401 it sends the user to the log-in page; on 403 it shows "not allowed". Mixing them up sends logged-in users to a log-in page that cannot help them.

Remembering who you are: sessions vs tokens

After logging in, the client must prove who it is on **every** request: either with a random **session id** in a cookie, which the server looks up, or with a signed **token** (a JWT) that carries the identity itself.

Server sessions vs JWT

	Server session	JWT
What the client holds	A random id that means nothing alone	A signed token with the user id, role and expiry inside
Server keeps	One row per active session	Nothing (only the signing secret)
Each request	Look the id up in the store	Verify the signature (no lookup)
Log out or ban now	Delete the row: immediate	Hard: the token works until it expires (or keep a denylist)
Several servers	Need a shared store (often Redis)	Work out of the box (same secret)
Size	About 30 characters	Hundreds of bytes, on every request
Usual transport	A cookie, sent automatically	An Authorization: Bearer header, added by your code

Example. Ana clicks "log out". With sessions, the server deletes row `s_8f3a...`, and the next request with the old cookie gets **401** immediately. With JWTs, the front end throws its copy of the token away; but if an attacker had copied it, it keeps working until it expires, up to an hour with `expiresIn: "1h"`.

Common mistake. "JWTs are more secure than sessions" (or the reverse). Neither is: they move the problem. A stolen JWT works like a password until it expires, and you cannot cancel it easily; a session needs a server-side store that every request must reach.

Passwords

Store a hash, never the password

A hash function turns a password into a fixed-length fingerprint that cannot be turned back into the password; the server stores only the fingerprint and checks a log-in by hashing the attempt again and comparing.

Three ways to keep a password, from worst to right

	Plaintext	Encrypted	Password hash (bcrypt)
Stored value	hunter2	Ciphertext, plus a key on the server	\$2b\$10\$... (60 characters)
Can it be turned back?	It is the password	Yes, by anyone with the key	No: it can only be guessed
Log-in check	Compare strings	Decrypt and compare	Hash the attempt and compare (<code>bcrypt.compare</code>)
After a database leak	Every password is known	Every password, if the key leaks too	Each password must be guessed, slowly

- **Why it matters:** databases leak (a stolen backup, an injection, a misconfigured admin panel), and people reuse passwords, so a plaintext leak also opens their email and bank accounts.
- **Same input, same output:** so the server can compare. **A tiny change, a completely different output:** so a near-miss reveals nothing.
- **Not any hash:** MD5, SHA-1 and SHA-256 are built to be fast, which helps the guesser. Password hashes (bcrypt, argon2) add a salt and a cost (see [Salt and cost](#)).
- **The hash is sensitive too:** never return it in a response, never put it in a token, and never log request bodies on `/auth` routes, or the plaintext password ends up in your log files.

Example. SHA-256 of hunter2 is `f52fbd32...26a3f6c7`; of hunter3 it is `fb8c2e2b...21cfa57a`: one character apart, nothing in common. A site whose "forgot password" email sends you **your old password** is storing it reversibly; the right flow emails a one-time reset link instead.

Common mistake. "Encrypting passwords with a secret key is safer than hashing." It is worse: the key has to live next to the data so the server can decrypt, and an attacker who gets the database usually gets the key too.

Salt and cost: why bcrypt is slow on purpose

A salt is a random value stored with each hash so that equal passwords get different hashes; a cost factor makes each hash deliberately slow, so an attacker can try far fewer guesses per second.



Each user gets their own salt; the cost sets the price of every guess.

bcrypt.hash(password, 10)

Generates a fresh random salt and embeds it, with the cost, in a 60-character result. One password_hash column is enough.

\$2b\$10\$...

\$2b\$ is the version, 10\$ the cost, then 22 characters of salt and 31 of hash.

The cost

An exponent: cost 10 means 2^{10} rounds, and every +1 **doubles** the time. OWASP asks for at least 10; pick the highest that keeps one log-in at a few hundred milliseconds, and raise it as hardware gets faster (old hashes keep working, because each stores its own cost).

bcrypt.compare(attempt, hash)

Re-hashes the attempt with the salt and cost stored in the hash: no separate salt needed.

argon2

OWASP's first choice today (argon2id): besides time it needs a lot of **memory** per guess, which hurts attackers using graphics cards. bcrypt remains a good default in Node; it uses only the first 72 bytes of a password.

```
// npm install bcrypt (bcryptjs has the same API)
const bcrypt = require('bcrypt');

// sign-up: a new random salt every time
const hash = await bcrypt.hash('correct horse battery staple', 10);
// e.g. '$2b$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcf17p921dGxad68LJZdL17lhWy'
// '$2b$' version · '10$' cost · then 22 chars of salt and 31 chars of hash

// log-in: re-hash the attempt with the salt and cost stored in the hash
await bcrypt.compare('correct horse battery staple', hash);
// → true
await bcrypt.compare('Correct horse battery staple', hash);
// → false
```

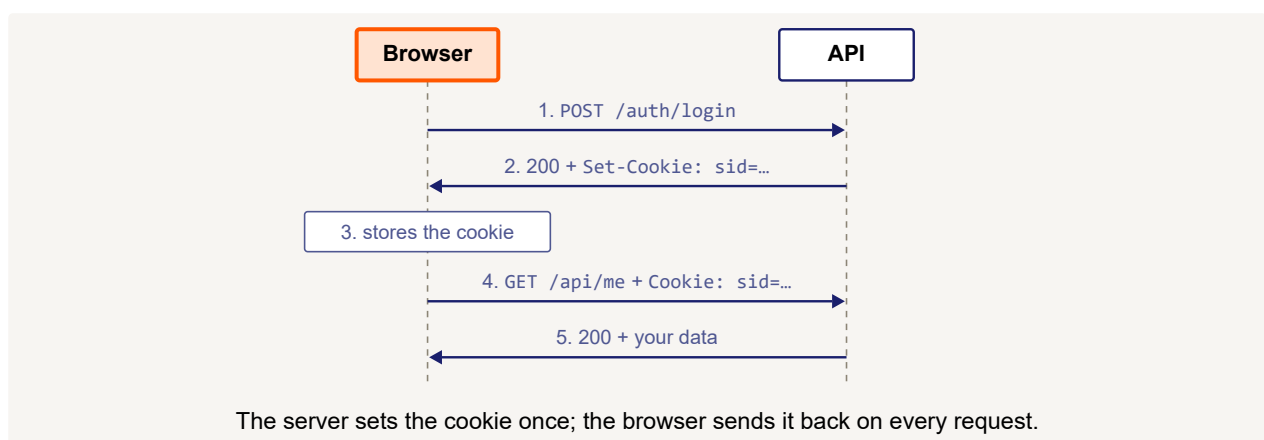
Example. Two users both choose 123456. Unsalted SHA-256 stores the same value twice, 8d969eef6ecad3c2..., and a web search for it reveals the password: crack one, crack both. With bcrypt the two rows look completely unrelated. In the password-hashing tool on this card, hash a password at 2^{16} and then 2^{18} iterations: the time roughly quadruples, and so does the attacker's.

Common mistake. Thinking the salt must be kept secret, or storing it "safely" elsewhere. A salt is not a secret: its only job is to be **different for every user**. Using one fixed salt for everybody defeats that job, because equal passwords become equal hashes again.

Tokens and cookies

Cookies and their security flags

A **cookie** is a small name=value that a server asks the browser to keep (Set-Cookie) and that the browser sends back automatically (Cookie); the flags **HttpOnly**, **Secure** and **SameSite** limit who can read it and when it is sent.



HttpOnly	JavaScript cannot read the cookie (<code>document.cookie</code> does not show it), so a script injected into the page (XSS, see textContent vs innerHTML) cannot steal it.
Secure	Sent only over HTTPS, never over plain HTTP.
SameSite	Whether the cookie goes along on requests started by another site. <code>Strict</code> : never. <code>Lax</code> (the default in modern browsers): only when the user navigates to your site, such as clicking a link, not on another site's background fetch or form POST. <code>None</code> : always, and then <code>Secure</code> is required.
Max-Age / Expires	How long the browser keeps it. Without them it is a "session cookie", gone when the browser closes.

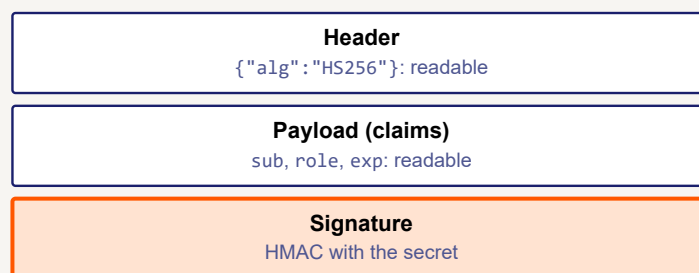
```
// Express 4: after a successful log-in with a server-side session
res.cookie('sid', sessionId, {
  // invisible to JavaScript
  httpOnly: true,
  // HTTPS only
  secure: true,
  // not sent on other sites' fetch / form POSTs
  sameSite: 'lax',
  // in milliseconds here (1 hour)
  maxAge: 60 * 60 * 1000,
});
// Response header:
// Set-Cookie: sid=8f3a...; Max-Age=3600; Path=/; Expires=...; HttpOnly; Secure; SameSite=Lax
```

Example. A forum shows comments with `innerHTML`, and someone posts ``. Without `HttpOnly`, every reader's session id is sent to the attacker, who can then act as them. With `HttpOnly`, `document.cookie` does not contain `sid` and the theft fails (the real fix is still to stop the XSS).

Common mistake. Thinking `HttpOnly` makes a cookie "safe". It stops scripts from **reading** it, but the browser still **sends** it, so CSRF is still possible (that is `SameSite`'s job), and an XSS can still make requests as the user from inside the page.

Anatomy of a JWT

A **JSON Web Token** is three `base64url` strings joined by dots, `header.payload.signature`: the header names the signing algorithm, the payload holds the **claims** (statements such as the user id and the expiry time), and the signature proves that the first two have not been changed.



Two readable parts and a signature that seals them.

Claims you will meet (a typical log-in token has `sub`, `role`, `iat` and `exp`)

Claim	Name	Meaning	Who sets it
<code>sub</code>	Subject	Who the token is about: the user id. Becomes <code>req.user.sub</code> .	You: <code>jwt.sign({ sub: user.id, ... })</code>
<code>iat</code>	Issued at	When it was signed.	<code>jwt.sign</code> , automatically
<code>exp</code>	Expiration	After this moment <code>jwt.verify</code> throws <code>jwt expired</code> .	<code>jwt.sign</code> with <code>expiresIn: "1h"</code> , which sets <code>iat + 3600</code>
<code>nbf</code>	Not before	Not valid before this moment.	Optional (<code>notBefore</code>)

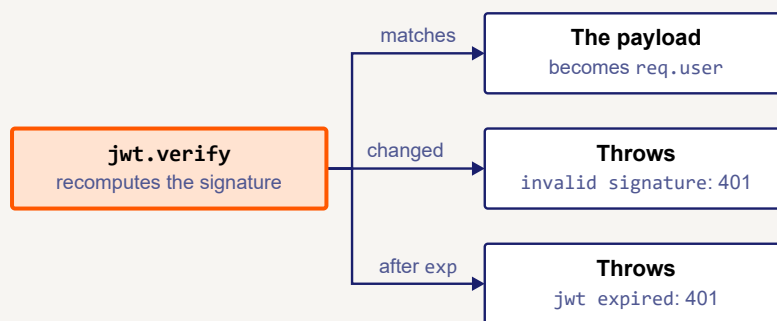
Claim	Name	Meaning	Who sets it
iss / aud	Issuer / audience	Who issued it / who it is meant for.	Optional
role	(your own)	Lets routes check permissions without a database lookup.	You

Example. The example token on jwt.io starts `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9`, which decodes to the header `{ "alg": "HS256", "typ": "JWT" }`. Its payload is `{ "sub": "1234567890", "name": "John Doe", "iat": 1516239022 }`: iat is 18 January 2018, 01:30:22 UTC. It has no exp, so it never expires, which you would never do in a real API. Paste it into the JWT inspector on this card and change one character.

Common mistake. Putting secrets in the payload ("it is a token, so it is hidden"): a password hash, another user's data, an internal API key. Anyone who sees the token can decode it in one line. A second classic: `exp: Date.now() + 3600`, which is in **milliseconds**, so the token expires in about 56,000 years.

Signing and verifying: what the secret does

With **HS256**, `jwt.sign` computes an **HMAC-SHA-256** of `header.payload` using a secret that only the server knows; `jwt.verify` recomputes it and rejects the token if the result differs, if the algorithm is not the expected one, or if the token has expired.



`jwt.verify` either returns the payload or throws: there is no "almost valid".

HMAC A hash mixed with a secret key.

HS256 HMAC with SHA-256. **Symmetric**: the same secret signs and verifies, so it never leaves the server (it lives in `process.env.JWT_SECRET`, see [The signing secret](#)).

RS256 Signs with a private key and verifies with a public key: for when other services must check tokens without being able to create them.

- **Signing is not encrypting**: a signature protects **integrity** (nobody changed it), not **confidentiality** (nobody can read it).
- `"alg": "none"` is an unsigned token. Libraries that trusted the header's `alg` once accepted forgeries this way. `jsonwebtoken 9` refuses them when you pass a secret; `{ algorithms: [ 'HS256' ] }` makes the rule explicit.
- **What `jwt.verify` throws**: `TokenExpiredError` (`jwt expired`), `NotBeforeError` (`jwt not active`) or `JsonWebTokenError` (`invalid signature, jwt malformed...`).

```

const jwt = require('jsonwebtoken');

// at log-in
const token = jwt.sign({ sub: user.id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '1h' });

// on every protected request
try {
  const payload = jwt.verify(token, process.env.JWT_SECRET, { algorithms: [ 'HS256' ] });
  // payload → { sub: 42, role: 'user', iat: 1791158400, exp: 1791162000 }
} catch (err) {
  // err.name → 'TokenExpiredError' | 'NotBeforeError' | 'JsonWebTokenError'
}
  
```

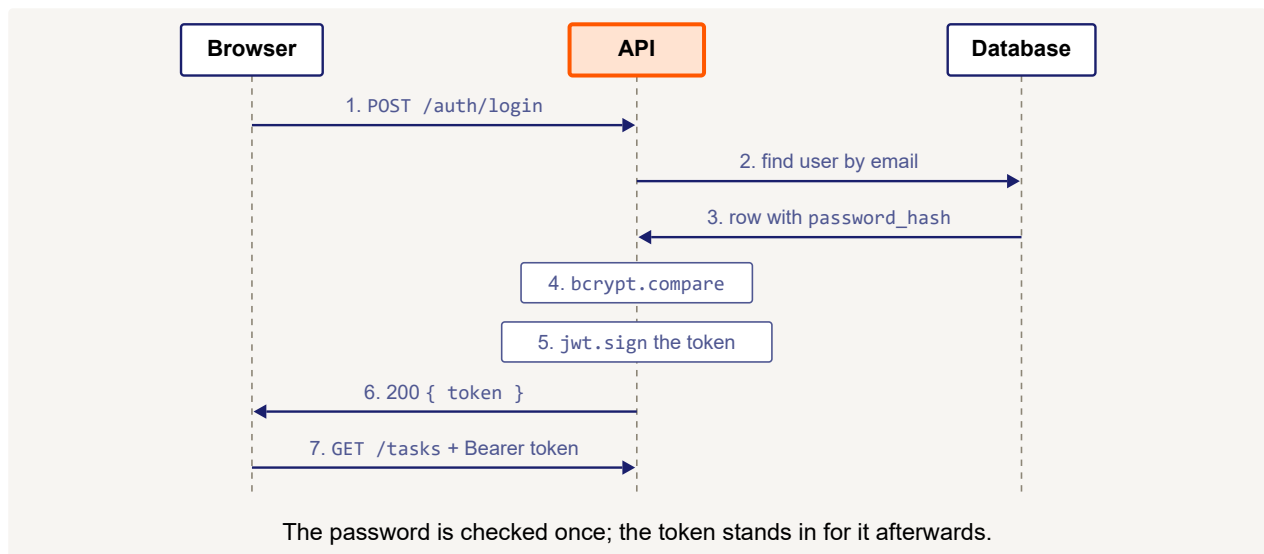
```
// A strong random secret (run once, put the output in .env):
// node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Example. Ana decodes her token, changes "role": "user" to "role": "admin", encodes it again and sends it. The server recomputes HMAC-SHA-256 over the new header and payload with its secret; the result does not match the old signature still attached, so `jwt.verify` throws `invalid signature` and the middleware answers **401**. Try it with the **Tamper** button of the inspector.

Common mistake. Using `jwt.decode(token)` in the middleware instead of `jwt.verify(token, secret)`. `decode` only reads the payload and checks nothing, so anyone can write a token saying `role: admin`. Another: a short secret such as `secret` or `riverside123`. Anyone holding one token can try millions of candidate secrets offline until the signature matches; use 32 random bytes.

Register and log in, step by step

Registering validates the input, hashes the password and stores the user; **logging in** finds the user by email, compares the password with `bcrypt.compare` and, if it matches, issues a signed token that the client sends on later requests.



Registering

1. Validate: email a non-empty string, password at least 8 characters; otherwise 400.
2. If the email already exists: 409 Conflict.
3. Hash with `bcrypt.hash(password, 10)` and store the user.
4. Answer 201 with `{ id, email }` only: never the hash.

Logging in

- **One answer for both failures:** no such user, or `bcrypt.compare` says false, both get 401 `Invalid credentials`. Different messages ("no such email" vs "wrong password") let an attacker discover which emails have accounts: **account enumeration**.
- **On success:** sign `{ sub: user.id, role: user.role }` with `expiresIn: "1h"` and answer 200 `{ token }`.
- **Both handlers are** `async` (the database and `bcrypt` return promises): in Express 4 an error after an `await` must reach `next(err)` (see [Async handlers](#)).

```
// The heart of the log-in check: one answer for both failure cases.
const user = await userModel.findByEmail(email);
// → null if unknown
const ok = user !== null && (await bcrypt.compare(password, user.password_hash));
if (!ok) return res.status(401).json({ error: 'Invalid credentials' });
// ... sign the token and answer 200 { token }
```

Example. Ana registers with `{"email": "ana@example.com", "password": "password1"}` → **201** `{"id": 1, "email": "ana@example.com"}`. Registering again → **409**. Logging in with password2 → **401** Invalid credentials; with an unknown email → the same **401**. With the right password → **200** `{"token": "eyJhbGciOiJIUzI1NiIs..."}`, which she then sends as Authorization: Bearer eyJ... on every /tasks request.

Common mistake. Calling `bcrypt.compare(password, user.password_hash)` without checking that `user` exists. For an unknown email `user` is `null`, reading `.password_hash` throws a `TypeError`, and the client gets `500` instead of `401` (many tutorial snippets have this bug). Also wrong: `password === user.password_hash`, which compares a password with a hash and is never true.

Sending and expiring tokens

The client sends the token on every protected request in the `Authorization: Bearer <token>` header, and the `exp` claim limits how long a stolen copy stays useful.

- **The header:** exactly Authorization: Bearer <token>, one space after Bearer.
- **Never in a URL:** ?token=... ends up in browser history and server logs.
- **Expiry is the built-in revocation:** one hour is a common choice. "Logging out" with a JWT means the client deletes its copy; larger apps pair a short-lived token with a revocable **refresh token** in an HttpOnly cookie.
- **Where a browser app keeps it** (memory, localStorage or an HttpOnly cookie) is a front-end trade-off: see Where to keep the token. A cookie also needs CORS with credentials.

```
// Front end: log in, keep the token, send it back
const login = await fetch('http://localhost:3000/auth/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email, password }),
});
if (!login.ok) throw new Error('Wrong email or password');
const { token } = await login.json();

const res = await fetch('http://localhost:3000/tasks', {
  headers: { Authorization: `Bearer ${token}` },
});
// From a terminal: curl -H "Authorization: Bearer $TOKEN" http://localhost:3000/tasks
```

Example. A token signed at 10:00:00 with `expiresIn: "1h"` has `exp = 11:00:00`. A request at 10:59:59 passes; at 11:00:00 exactly `jwt.verify` throws `jwt expired` (the check is "`now ≥ exp`"), the API answers **401**, and the front end should send the user back to the log-in form. Shift the clock in the JWT inspector to watch it happen.

Common mistake. Getting the header format wrong: `Authorization: eyJ...` (no `Bearer`), `Bearer: eyJ...` (a header called `Bearer`), or `?token=eyJ...` in the URL. A typical auth middleware only accepts a value that starts with `Bearer` followed by the token, and treats anything else as a missing token (401).

Protecting routes

The auth middleware: one gate in front of many routes

An auth middleware runs before the protected routes: it reads the Authorization header, verifies the token, stores the decoded payload in req.user and calls next(), or stops the request with 401.

Three requests to GET /tasks

Request header	What auth does	Result
(none)	token is null	401 Missing token; the router never runs
Authorization: Bearer abc	jwt.verify throws jwt malformed	401 Invalid or expired token

Request header	What auth does	Result
Authorization: Bearer eyJ... (valid)	req.user = { sub: 42, role: 'user', iat, exp }, then next()	The tasks controller runs with req.user

- **Leave the door open where tokens come from:** /auth/register and /auth/login are mounted **without** auth.
- cors **before** auth: the browser's preflight never carries the token (see [Preflight requests](#)).
- **Only 401 here:** the middleware knows who you are, not what you may do. The 403 decisions belong to the controllers or a role middleware placed after it (see [Ownership and roles](#)).
- **A plain try/catch is enough:** jwt.verify without a callback is synchronous.

```
// middleware/auth.js
const jwt = require('jsonwebtoken');

function auth(req, res, next) {
  // Node lower-cases header names
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'Missing token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET, { algorithms: ['HS256'] });
    next();
  } catch (err) {
    res.status(401).json({ error: 'Invalid or expired token' });
  }
}

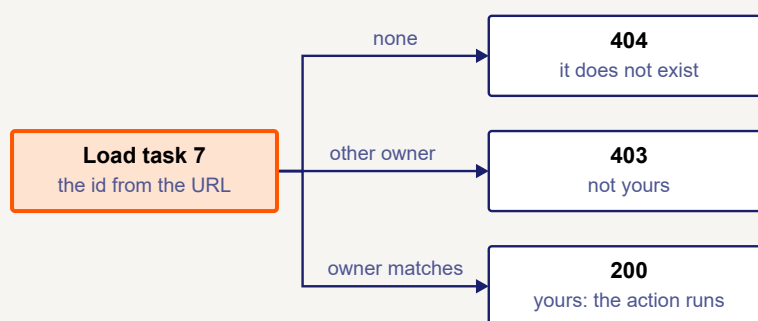
// app.js
// open: this is where tokens come from
app.use('/auth', authRouter);
// every /tasks route is behind the gate
app.use('/tasks', auth, tasksRouter);
```

Example. Ana's React page calls GET /tasks?page=2 with her token. auth verifies it and sets req.user.sub = 1; the list controller then asks the model for **her** tasks only (findByOwner(1, { limit, offset })). Ben's token on the same URL gives Ben's tasks: same route, different req.user.

Common mistake. Mounting the gate on the whole app, app.use(auth), above the log-in route. Now /auth/login itself demands a token, so nobody can ever get one: every log-in answers **401** Missing token. Mount auth on the routers it protects, after the open /auth router.

Ownership and roles: a valid token is not a free pass

An **ownership check** compares the user in the verified token (req.user.sub) with the owner stored on the resource before reading or changing it; a **role check** asks whether the user's role (for example admin) permits the action at all.



Load first, then compare owners: the answer depends on whose resource it is.

- **The owner always comes from the verified token**, never from the body, query or URL: on create you store `req.user.sub`, and a `userId` sent by the client is ignored.
- **Every read, update and delete of one resource loads it and compares owners** before doing anything: 404 if it does not exist, 403 if it is not yours.
- **Lists filter in the database query** (`findByOwner(req.user.sub, ...)`), so other people's rows never leave the database.
- **Compare values of the same type**: a JWT can carry `sub` as a number (42) while `pg` returns a `BIGINT` column as a string ("42"), and `42 !== "42"`. Write `Number(task.user_id) !== Number(req.user.sub)`.
- **Roles add a second layer**: the token also carries `role` (user or admin), and a middleware such as `requireRole('admin')` refuses a whole route to everyone else with 403.

```
// A notes API (the same pattern works for tasks): load, 404, then 403.
async function getNote(req, res, next) {
  try {
    const note = await notesModel.findById(Number(req.params.id));
    if (!note) return res.status(404).json({ error: 'Note not found' });
    if (Number(note.owner_id) !== Number(req.user.sub)) {
      return res.status(403).json({ error: 'This note is not yours' });
    }
    res.json(toDTO(note));
  } catch (err) {
    next(err);
  }
}

// A role check: a middleware factory used after auth.
const requireRole = (role) => (req, res, next) =>
  (req.user.role === role ? next() : res.status(403).json({ error: 'Forbidden' }));

router.delete('/:id', requireRole('admin'), deleteNote);
```

Example. Ana (`sub: 1`) sends `PUT /notes/8`; note 8 has `owner_id: 2`. Her token is perfect, so auth lets her through, and the controller answers **403**. She then sends `POST /notes` with the body `{ "title": "Hi", "userId": 2 }`: the server ignores `userId` and stores the note with owner 1, the `sub` from her token.

Common mistake. Taking the owner from the request: `create({ ...req.body })` or `findByOwner(req.query.userId)`. Anything the client sends can be changed by the client; only the verified token says who is calling. A second classic: comparing ids with `!==` without converting them, so every owner gets 403 (or, with `!=`, getting lucky without understanding why).

CORS

Origins and the same-origin policy

An **origin** is the scheme + host + port of a URL; the browser's **same-origin policy** lets a page's JavaScript read responses freely only from its own origin.

Compared with `http://localhost:3000/api/tasks`

URL	Same origin?	Why
<code>http://localhost:3000/index.html</code>	Yes	Only the path differs, and the path does not count
<code>http://localhost:5173/</code>	No	Port 5173 $\neq$ 3000
<code>https://localhost:3000/</code>	No	Scheme https $\neq$ http
<code>http://127.0.0.1:3000/</code>	No	Host 127.0.0.1 $\neq$ localhost (same machine, different name)
<code>http://localhost/</code>	No	No port written means 80 $\neq$ 3000 (see URL anatomy)

- **What it blocks:** JavaScript **reading** a cross-origin response (`fetch`).
- **What it does not block:** embedding other origins (`<img>`, `<script src>`, stylesheets), submitting forms and following links to them: that is how the web has always worked.

- **Many cross-origin requests are still sent:** the browser just refuses to hand the answer to the script.
- **The usual development setup is cross-origin:** React on localhost:5173 calling Express on localhost:3000, which is why the API needs **CORS headers**. Serving the page from Express itself with `express.static` gives page and API one origin.

Example. The page at `http://localhost:5173` runs `fetch('http://localhost:3000/api/tasks')`. The ports differ, so it is cross-origin. With no CORS configured, the Express terminal still logs `GET /api/tasks: the request arrived and the route ran`. But the browser console shows a CORS error and the page's `fetch` rejects with `TypeError: Failed to fetch`.

Common mistake. "The same-origin policy stops the request from reaching the server." For many requests it does not: the server receives and runs them, and only the response is hidden from the page. That is one more reason why a GET must never change data, and why CORS settings are no replacement for authentication.

CORS: the server says who may read

CORS (Cross-Origin Resource Sharing) is a set of HTTP response headers with which a server tells the browser which other origins may read its responses; the key one is `Access-Control-Allow-Origin`.

Simple request One an old-fashioned HTML form could already send: method GET, HEAD or POST; only "safelisted" headers (such as `Accept`); and, with a body, a Content-Type of `application/x-www-form-urlencoded`, `multipart/form-data` or `text/plain`. The browser sends it straight away and checks the CORS headers on the answer.

Any other request A PUT, a JSON body, an Authorization header: the browser first asks permission with a preflight.

- The **cors package** is not part of Express: `npm install cors`.
- `app.use(cors())` **with no options** sends `Access-Control-Allow-Origin: *` on every route: fine for a public, read-only API, too wide for one with user accounts.
- **Pass** origin with the exact front-end origin, or an array of them, and mount it **before** the routers.

```
// npm install cors
const cors = require('cors');

// before the routers (and before auth)
app.use(cors({ origin: 'http://localhost:5173' }));
app.use(express.json());
app.use('/api/tasks', tasksRouter);

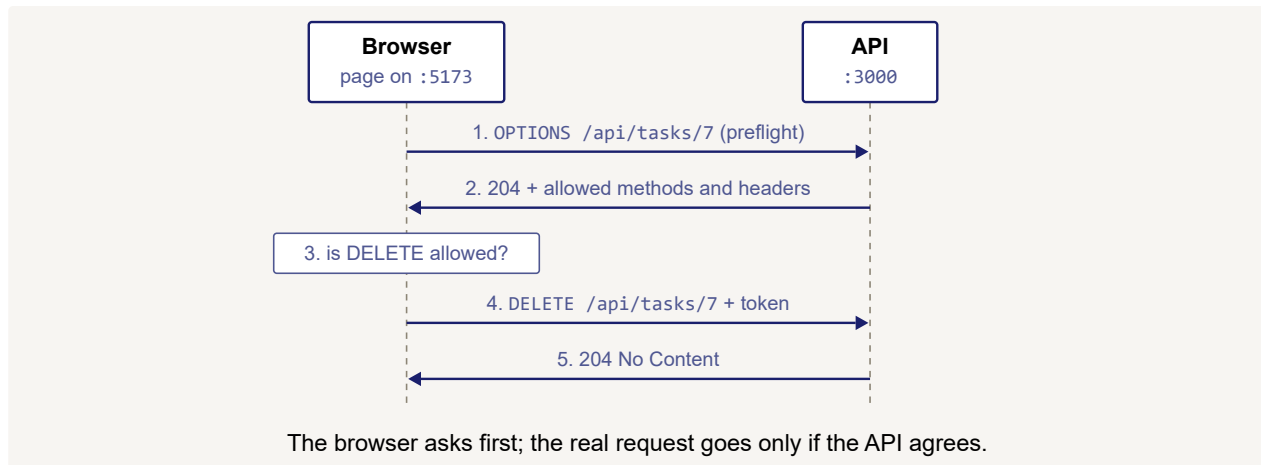
// The exchange it produces:
// GET /api/tasks HTTP/1.1
// Origin: http://localhost:5173
//
// HTTP/1.1 200 OK
// Access-Control-Allow-Origin: http://localhost:5173
// Vary: Origin           ← caches must keep one copy per Origin
// Content-Type: application/json; charset=utf-8
```

Example. With `cors({ origin: 'http://localhost:5173' })`, the React page reads the tasks. A page on `https://evil.example` making the same call gets `Access-Control-Allow-Origin: http://localhost:5173`, which does not match its own origin, so the browser blocks it with the message "...has a value 'http://localhost:5173' that is not equal to the supplied origin". Try both in the CORS simulator on this card.

Common mistake. Adding `Access-Control-Allow-Origin` to the **request** in the front-end code: `fetch(url, { headers: { 'Access-Control-Allow-Origin': '*' } })`. It is a **response** header that only the server can grant; on a request it does nothing useful, and as a non-safelisted header it even triggers a preflight.

Preflight requests

Before a cross-origin request that is not "simple" (for example a PUT, a DELETE, a JSON body or an Authorization header), the browser first sends an OPTIONS **preflight** asking for permission, and sends the real request only if the answer allows it.



When the browser preflights

- **The method** is not GET, HEAD or POST;
- **or a header** other than Accept, Accept-Language, Content-Language and Content-Type is sent (Authorization always counts);
- **or** Content-Type is anything other than the three form types (application/json counts).
- **Cookies** (credentials) do not cause a preflight on their own.

The question (OPTIONS) Origin, Access-Control-Request-Method (the method it wants to use) and Access-Control-Request-Headers (the non-safelisted headers it wants to send).

The answer A 2xx status with a matching Access-Control-Allow-Origin, the method in Access-Control-Allow-Methods and every requested header in Access-Control-Allow-Headers. Access-Control-Max-Age lets the browser reuse the permission for some seconds. The cors middleware answers preflights by itself with 204.

HTTP

```

OPTIONS /api/tasks/7 HTTP/1.1
Host: localhost:3000
Origin: http://localhost:5173
Access-Control-Request-Method: DELETE
Access-Control-Request-Headers: authorization

HTTP/1.1 204 No Content
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
Access-Control-Allow-Headers: authorization
Access-Control-Max-Age: 600
Vary: Origin, Access-Control-Request-Headers
  
```

Example. The React page runs `fetch('http://localhost:3000/api/tasks/7', { method: 'DELETE', headers: { Authorization: 'Bearer ...' } })` against `cors({ origin: 'http://localhost:5173', methods: ['GET', 'POST'] })`. The preflight answer lists only GET,POST, so Chrome logs "Method DELETE is not allowed by Access-Control-Allow-Methods in preflight response.", the DELETE is never sent and the task is still there.

Common mistake. Mounting the auth middleware before cors. The browser never sends Authorization on a preflight, so auth answers the OPTIONS request with 401; a preflight that is not 2xx fails, and every protected call from the front end shows a CORS error even though the token is fine. Put `app.use(cors(...))` first.

Cookies across origins: credentials

By default `fetch` **does not send cookies to another origin; to send them the page must ask with** `credentials: 'include'`, **and the server must answer with an exact origin (never *)** plus `Access-Control-Allow-Credentials: true`.

- **The page** uses `credentials: 'include'`.
- **The response** has `Access-Control-Allow-Origin` equal to the page's origin and `Access-Control-Allow-Credentials: true`; on a preflight, `*` in the allowed methods or headers no longer counts as a wildcard either. In Express: `cors({ origin: 'http://localhost:5173', credentials: true })`.
- **Never** `origin: true` **with credentials**: it echoes back whatever origin asks, so every website could read your users' data with their cookies.
- **A Bearer token in a header** instead of cookies needs no credentials at all.
- **Same site, different origin**: `localhost:5173` and `localhost:3000` are different origins but the same **site** (a site ignores the port), so `SameSite=Lax` cookies still flow in development; on different domains in production you need `SameSite=None; Secure`.

```
// Server
app.use(cors({ origin: 'http://localhost:5173', credentials: true }));

// Page
const res = await fetch('http://localhost:3000/api/me', { credentials: 'include' });
```

Example. With plain `cors()` and `credentials: 'include'`, Chrome logs: "The value of the 'Access-Control-Allow-Origin' header in the response must not be the wildcard '*' when the request's credentials mode is 'include'." Changing the server to an exact origin and `credentials: true` fixes it. Challenge 5 of the simulator is exactly this.

Common mistake. `cors({ origin: '*', credentials: true })`. The `cors` package sends exactly what you asked for, `*` together with `Allow-Credentials: true`, and the browser rejects that combination, so nothing works and the configuration looks "correct" in the code.

What CORS is not

CORS is enforced only by browsers and only controls which web pages may read responses; it does not identify anyone, and it does not stop curl, Postman, scripts or other servers from calling your API.

Who CORS applies to

Client	Sends Origin?	Checks CORS headers?	Can call your API?
Your React page (another origin)	Yes	Yes	Yes, and reads the answer if CORS allows it
A page on another site	Yes	Yes	Simple requests are sent and run; reading is blocked
curl, Postman, a script	No	No	Yes, always
Another server (Node <code>fetch</code>)	No	No	Yes, always

- **"It works in Postman but not in React"**: the API is fine; the browser is blocking the page because of CORS.
- **CORS can only relax the same-origin policy**: adding `cors()` never makes an API safer, and a careless configuration (`origin: true` with `credentials: true`) makes it less safe.
- **Simple requests run before the browser checks the answer**, so a cross-site form POST still runs on your server: protecting state-changing routes is the server's job (auth, `SameSite` cookies).

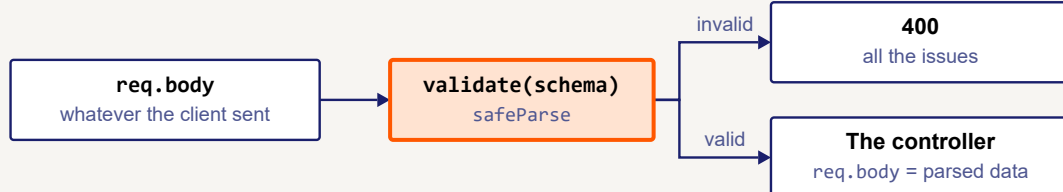
Example. Your API uses `cors({ origin: 'http://localhost:5173' })` but forgot the auth middleware on `DELETE /api/tasks/:id`. Anyone can run `curl -X DELETE http://localhost:3000/api/tasks/7` and the task is gone: the CORS setting played no part.

Common mistake. "I restricted CORS to my front end, so only my front end can use my API." Anyone can still call it outside a browser, and even inside a browser other sites' simple requests reach the server. Only authentication and authorisation decide who may do what.

Hardening the API

Validation with a schema: Zod and a validate middleware

Input validation checks every value that arrives over HTTP against a declared shape before your code uses it; with **Zod** you declare the shape once as a schema, and one `validate(schema)` middleware rejects bad input with **400** before any controller runs.



Bad input stops at the gate; the controller only ever sees clean data.

- `z.object({ ... })`** The schema: each field with its type and rules (`z.string().trim().min(1)`, `z.boolean().default(false)`). Zod version 3 in these examples (npm install zod).
- `schema.safeParse(value)`** Never throws: returns `{ success: true, data }` or `{ success: false, error }`.
- `error.issues`** An array of problems, each with a path (which field), a message and a code.
- Unknown keys** Object schemas **strip** them by default, so `userId` and `isAdmin` disappear from data; `.strict()` rejects them instead.

```

const { z } = require('zod');

const taskSchema = z.object({
  title: z.string().trim().min(1).max(120),
  done: z.boolean().default(false),
});

taskSchema.safeParse({ title: '', done: 'yes', userId: 7 });
// → { success: false, error } with error.issues:
// [ { code: 'too_small', path: ['title'], message: 'String must contain at least 1 character(s)', ... },
//   { code: 'invalid_type', path: ['done'], message: 'Expected boolean, received string', ... } ]

taskSchema.safeParse({ title: ' Buy milk ', userId: 7 });
// → { success: true, data: { title: 'Buy milk', done: false } }  userId stripped, default added

// middleware/validate.js: one middleware for every schema
function validate(schema) {
  return (req, res, next) => {
    const result = schema.safeParse(req.body);
    if (!result.success) return res.status(400).json({ error: result.error.issues });
    // continue with the parsed data only
    req.body = result.data;
    next();
  };
}

// routes/tasks.js: every route that writes
router.post('/', validate(taskSchema), createTask);
router.put('/:id', validate(taskSchema), replaceTask);
  
```

Example. POST /tasks with {"done":true} (no title) never reaches the controller: validate(taskSchema) answers 400 {"error":[{"code":"invalid_type","path":["title"],"message":"Required",...}]}. With {"title":"Buy milk","userId":99} it passes, req.body becomes { title: 'Buy milk', done: false }, and the controller sets the owner from req.user.sub.

Common mistake. Validating and then carrying on with the original req.body instead of the parsed data: the stripped fields (userId, isAdmin) come straight back. Another: validating POST but forgetting PUT and PATCH, which write to the same database.

Security middleware: helmet, rate limits, body limits, HTTPS

A few lines of middleware close whole classes of holes: **helmet** sets protective response headers, **express-rate-limit** caps how many requests one client may send, **express.json({ limit })** caps body size, and **HTTPS with HSTS** keeps passwords and tokens from crossing the network in plain text.

What each one protects against

Middleware	Protects against	When it triggers
helmet()	MIME sniffing, clickjacking, some XSS, revealing that you run Express	Adds headers to every response
rateLimit(...)	Password guessing, scrapers, one client flooding the API	429 Too Many Requests
express.json({ limit })	Huge bodies that exhaust memory	413 Payload Too Large
cors({ origin })	Other web pages reading your API in a browser	Missing CORS headers: the browser blocks
HTTPS + HSTS	Passwords and tokens read on the network	Always (with HSTS the browser refuses plain http)

helmet() Sets X-Content-Type-Options: nosniff, Content-Security-Policy (which scripts may run), Strict-Transport-Security and X-Frame-Options (no embedding your pages in another site's frame), and removes X-Powered-By: Express.

express-rate-limit Answers 429 once one IP address exceeds a limit; a strict one on /auth/login stops password guessing.

Body limit The body parser already rejects bodies over 100kb with 413 (see [express.json\(\)](#)): the point is never to raise it carelessly.

HTTPS and HSTS HTTPS encrypts everything, including Authorization; in production the hosting platform or a proxy usually handles it. HSTS (Strict-Transport-Security: max-age=...) tells browsers to use only HTTPS for your domain from then on.

```
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');

app.use(helmet());
// the default: keep it small
app.use(express.json({ limit: '100kb' }));
app.use('/auth/login', rateLimit({ windowMs: 15 * 60 * 1000, limit: 10 }));
// → 10 tries / 15 min / IP
app.use(rateLimit({ windowMs: 60 * 1000, limit: 100 }));
// → 100 requests / min / IP
// express-rate-limit 7 calls the option `limit`; older versions and many tutorials call it `max`.
```

Example. An attacker scripts 10,000 log-in attempts for ana@example.com. With the log-in limiter, attempt 11 within 15 minutes gets 429; at 10 tries per quarter of an hour, 10,000 guesses take more than ten days, and bcrypt makes each one slow on top. Without it, the only limit is your server's CPU.

Common mistake. Thinking helmet() "secures the API". It only sets response headers: it does nothing against broken access control, injection or a leaked secret. Security middleware is a seatbelt, not a substitute for driving carefully.

The signing secret: leaks and fallbacks

Whoever holds `JWT_SECRET` can sign a valid token for any user with any role, so the secret lives only in the server's environment and is changed (rotated) the moment it may have leaked.

- **A leaked secret signs admin tokens:** { sub: 1, role: 'admin' }, signed in seconds with the leaked value, passes `jwt.verify` on your server.
- **Rotation logs everyone out:** after you change the secret, every existing token stops verifying and every user logs in again. That is the price, and it is worth paying.
- **A committed secret is a leaked secret**, even if a later commit deletes it: Git keeps the history.
- **The development fallback trap:** example projects write `process.env.JWT_SECRET || 'dev-secret-change-me'` so they run with zero setup. In production, a missing variable silently falls back to a secret printed in a public repository: fail fast there instead.

```
// .env (in .gitignore, never committed)
// JWT_SECRET=3f9c2b7e0d8a41f6b5c3e9a7d2f1084c6b5a3e2d1f0c9b8a7e6d5c4b3a291807

// config.js
// or start Node 20.6+ with: node --env-file=.env src/server.js
require('dotenv').config();
const isProduction = process.env.NODE_ENV === 'production';
if (isProduction && !process.env.JWT_SECRET) {
  // crash at start-up, not on the first log-in
  throw new Error('JWT_SECRET is missing');
}
module.exports = { jwtSecret: process.env.JWT_SECRET || 'dev-secret-change-me' };
```

Example. A pair pushes `const JWT_SECRET = 'riverside123';` to a public repository. Anyone can read it, sign a token saying role: admin and delete every fixture. The fix: generate a new random secret, put it in the server's environment, remove it from the code, and accept that every user has to log in again.

Common mistake. Treating the fallback as harmless "because it is only for development": if `JWT_SECRET` is missing in production, the server signs with the public fallback and anyone who has read the repository can sign admin tokens. The other quiet leak is logging `process.env` or the whole config at start-up, which prints the secret into log files.

Pagination limits as protection

A list endpoint returns one page at a time, chosen by `?page=` and `?limit=`, and the server **clamps** both to safe bounds, so no caller can make it read and send an unbounded number of rows.

- **Query values arrive as strings**, or missing, or as arrays (`?limit=1&limit=2`; see [req.query](#)): convert with `Number`, and fall back to a default when the result is not a usable number.
- **Clamp both bounds:** round to an integer, then $\text{page} \geq 1$ and $1 \leq \text{limit} \leq 100$; the server computes `offset = (page - 1) * limit`.
- **In SQL** the page becomes `LIMIT $1 OFFSET $2`, always with an `ORDER BY` and always parameterised (see [Pagination in SQL](#)).
- **Huge offsets still cost work**, because the database skips that many rows: very large tables use **cursor pagination** ("the 20 after id 4520") instead.

```
function clampPage(query) {
  const page = Math.max(1, Math.floor(Number(query.page)) || 1);
  const limit = Math.min(100, Math.max(1, Math.floor(Number(query.limit)) || 20));
  return { page, limit, offset: (page - 1) * limit };
}

// nothing sent: defaults
console.log(clampPage({}));
// query values are strings
console.log(clampPage({ page: '3', limit: '10' }));
// both clamps at work
console.log(clampPage({ page: '0', limit: '999' }));
```



```
// negative values
console.log(clampPage({ page: '-4', limit: '-1' }));
// junk and decimals
console.log(clampPage({ page: 'abc', limit: '2.7' }));

// A common snippet with no lower bound for limit:
const naiveLimit = (q) => Math.min(100, Number(q.limit) || 20);
console.log(naiveLimit({ limit: '-1' }));
// → -1 would reach SQL as LIMIT -1
```

Example. GET /tasks?limit=999999 gives 100 items, ?page=0 is treated as page 1, and ?limit=abc falls back to 20. With the naive snippet, ?limit=-1 passes -1 to the database: PostgreSQL rejects LIMIT -1 with an error (a 500 for the client) and SQLite treats it as "no limit at all".

Common mistake. Clamping only the upper bound (as many tutorial snippets do), or using req.query.limit directly. Remember that Number('0') || 20 is 20 (0 is falsy) and that Number('-5') is a perfectly truthy -5.

The security mistakes small APIs actually make

Most breaches of small APIs come from a handful of mistakes that the **OWASP Top 10** lists every few years: broken access control, injection, exposed secrets, weak authentication and careless configuration. Each one has a known, simple defence.

Risk → how it looks → defence

Risk	In a small API	Defence	Where
Broken access control	GET /tasks/8 returns another user's task; the owner comes from the body	Ownership check with the owner from the token; filter lists by owner	Ownership and roles
Injection (SQL)	String concatenation in a query	Parameterised queries (\$1, ?)	SQL injection
Injection (NoSQL)	{"email":{"\$ne":null}} passed into a MongoDB find	Validate types with a schema	Validation with a schema
Cross-site scripting (XSS)	API data inserted with innerHTML	textContent; React escapes by default	textContent vs innerHTML
Exposed secrets or data	JWT_SECRET in Git; stack traces or password_hash in responses	Environment variables, rotation, generic error messages, a DTO	The signing secret
Authentication failures	Plaintext or fast hashes, no rate limit, tokens without exp, jwt.decode, alg: none	bcrypt, a log-in limiter, expiresIn, jwt.verify with algorithms	The Passwords and Tokens groups
Security misconfiguration	cors({ origin: true, credentials: true }), X-Powered-By, debug output	Exact CORS origins, helmet	Security middleware
Vulnerable dependencies	Old packages with known holes	npm audit, keep package-lock.json, update	package-lock.json

Example. A quick self-audit of a Tasks API: log in as Ana and request Ben's task (expect **403**); send a task with userId of someone else (expect your own id stored); send ?limit=-1 and ?limit=999999 (expect 1 and 100 items at most); search the repository for JWT_SECRET = (expect only process.env); trigger an error (expect no stack trace in the response).

Common mistake. "Security is something we add at the end." Some layers (helmet, rate limits) can be added at the edges, but the important decisions shape the code from day one: whether the owner comes from the token, whether passwords are hashed, whether errors leak details. Retrofitting them after users and data exist is far harder.